

Working with Arrays and JOIN Queries in Azure Cosmos DB

Summary:-

In this lab, you will work with JSON documents that contain arrays of objects in Azure Cosmos DB. You will create documents with nested arrays, execute SQL queries using the JOIN operator to retrieve array elements, and filter the results based on values stored within the array. This lab demonstrates how Azure Cosmos DB stores and queries complex JSON documents.

Let's Start:-

After completing this lab, you will be able to:

- Create JSON documents containing arrays of objects.
- Store array-based documents in an Azure Cosmos DB container.
- Query array elements using the JOIN operator.
- Retrieve nested properties from array objects.
- Filter documents based on values stored inside arrays.

Lab Steps:-

Step 1 – Create Documents with Arrays

- **Reference:** This lab uses the Azure Cosmos DB account, CosmosOrdersDb database, and Orders container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure the account with the required settings:
Workload type: Learning
Region: EAST US/ EAST US 2
Capacity Mode: Provisioned Throughput
Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Data Explorer**. Click on + and select **New Database**.
- Enter the name and click **ok**.
- Click on the three dots placed on the right side of created database. Click on **New Container**.
- Enter:
 - **Container ID:** Orders
 - **Partition Key:** /customerId
 - **Container Throughput:** Manual

- **Request Units (RU/s): 400**
- Click **Ok**. Expand the **Orders** container.
- Click **New Item**.
- Replace the default JSON with the provided order document containing an **Items** array.

Microsoft Azure | Search resources, services, and docs (G+)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

Azure Cosmos DB account

New Item Save Discard Upload Item

Orders.Items

SELECT * FROM c Type a query predicate (e.g., WHERE c.id="1"), or choose one from the drop down list, or leave empty to query all documents. Apply Filter

id	/customerid
ord-2001	C003

```

1 {
2   "id": "ord-2001",
3   "customerId": "C003",
4   "customerName": "Aisha",
5   "orderDateUtc": "2026-02-11T13:10:00Z",
6   "status": "Paid",
7   "items": [
8     { "courseId": "AZ-204", "courseName": "AZ-204 Azure Developer", "price": 49.99 },
9     { "courseId": "AZ-305", "courseName": "AZ-305 Azure Architect", "price": 69.99 }
10  ],
11  "totalAmount": 119.98,
12  "currency": "USD"
13 }
```

- Click **Save**. Click on **New Item** and save.

Microsoft Azure | Search resources, services, and docs (G+)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

Azure Cosmos DB account

New Item Save Discard Upload Item

Orders.Items

SELECT * FROM c Type a query predicate (e.g., WHERE c.id="1"), or choose one from the drop down list, or leave empty to query all documents. Apply Filter

id	/customerid
ord-2001	C003
ord-2002	C001

```

1 {
2   "id": "ord-2002",
3   "customerId": "C001",
4   "customerName": "Mark",
5   "orderDateUtc": "2026-02-11T14:00:00Z",
6   "status": "Paid",
7   "items": [
8     { "courseId": "AI-900", "courseName": "AI-900 Azure AI Fundamentals", "price": 19.99 },
9     { "courseId": "AZ-900", "courseName": "AZ-900 Azure Fundamentals", "price": 14.99 },
10    { "courseId": "DP-700", "courseName": "DP-700 Data Engineering", "price": 54.99 }
11  ],
12  "totalAmount": 89.97,
13  "currency": "USD"
14 }
```

- Repeat the process to add another document with an **Items** array.

Microsoft Azure | Search resources, services, and docs (G+)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

SELECT * FROM c | Type a query predicate (e.g., WHERE c.id="1"), or choose one from the drop down list, or leave empty to query all documents.

id	/customerId
ord-2001	C003
ord-2002	C001

```

1 {
2   "id": "ord-2002",
3   "customerId": "C001",
4   "customerName": "Mark",
5   "orderDateUtc": "2026-02-11T14:00:00Z",
6   "status": "Paid",
7   "items": [
8     {
9       "courseId": "AI-900",
10      "courseName": "AI-900 Azure AI Fundamentals",
11      "price": 19.99
12     },
13     {
14       "courseId": "AZ-900",
15       "courseName": "AZ-900 Azure Fundamentals",
16       "price": 14.99
17     }
18   ]
19 }

```

- Verify that both documents appear under **Items**.
- This step stores JSON documents containing arrays of objects.

Step 2 – Query Data from Nested Arrays

- Click **New SQL Query**.
- Enter the provided SQL query.
- Use the **JOIN** operator to retrieve:
 - Order ID
 - Customer ID
 - Course Name

Microsoft Azure | Search resources, services, and docs (G+)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

Search

New SQL Query

Execute Query

1 SELECT o.id, o.customerId, o.courseId, i.courseName FROM Orders o

2 JOIN i in o.items

Results

"id": "ord-2001",	"customerId": "C003",	"courseName": "AZ-204 Azure Developer"
"id": "ord-2001",	"customerId": "C003",	"courseName": "AZ-305 Azure Architect"
"id": "ord-2002",	"customerId": "C001",	

- Click **Execute Query**.
- Verify that the query returns the course details stored inside the **Items** array.
- This step retrieves data from nested arrays using the JOIN operator.

Step 3 – Filter Array Data

- Modify the SQL query to filter documents by Course ID.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

cf-nancy-az@behalrites...
MY DIRECTORY (BEHALRITESHG...)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

Search

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Quick start

Data Explorer

Mirroring in Fabric

Container Copy

Resource visualizer

Settings

Integrations

AI Capabilities (Preview)

+ New...

Search databases only

Home

CourseOrdersDb

Orders

Execute Query

Save Query

Download Query

View

Orders.Items

Orders.Quer...

```
1 SELECT o.id, o.customerId, o.courseId, i.courseName FROM Orders o
2 JOIN i in o.items
3 WHERE i.courseId = "AZ-305"
```

Results

Query Stats

Index Advisor

1 - 1

```
{
  "id": "ord-2001",
  "customerId": "C003",
  "courseName": "AZ-305 Azure Architect"
}
```

- Click **Execute Query**.
- Verify that only the matching order is returned.
- This step filters documents based on values stored within array elements.

Verification

Verify the following:

- Documents containing arrays are created successfully.
- The JOIN query retrieves data from nested array objects.
- Course ID and Course Name are displayed correctly.
- The filtered query returns only the document containing the specified Course ID.
- All SQL queries execute successfully without errors.